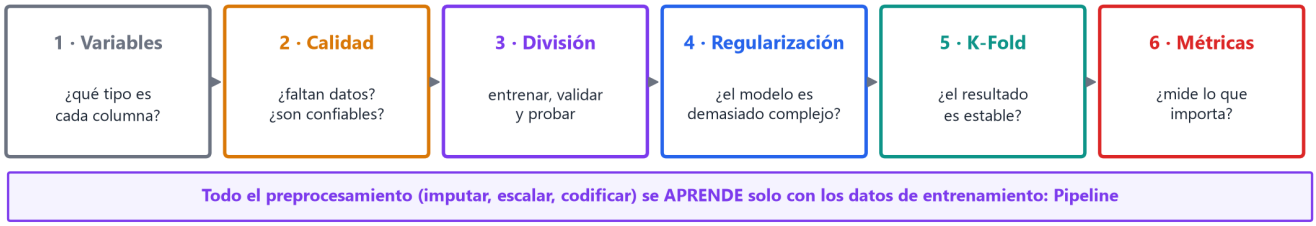
Clase 30 · Preparación para Machine Learning: variables, datos y validación

Profesora Laura Sánchez · inicio del bloque de Machine Learning · precede a Introducción al ML (C31)

La gran idea de la clase

En Machine Learning el modelo se juzga por **cómo se comporta con datos que nunca vio**. Por eso hay reglas del juego antes de cualquier algoritmo: conocer el **tipo** de cada variable, revisar la **calidad** de los datos, **dividir** en entrenamiento / validación / prueba, **regularizar** sobre variables comparables, **validar de forma estable** (K-Fold, estratificación) y elegir **métricas** acordes al costo de equivocarse. «Si entendemos las piezas, podremos armar el engranaje.»

El camino de los datos en una imagen



Lo que necesitas ANTES

- Regresión lineal y R^2 (Clases 17–19); regresión logística (Clases 20–21)
- Media, mediana, moda y desviación estándar (Etapa 0)
- Probabilidad condicional: precision y recall solo cambian el «total» (explicación de la clase)
- Python básico con pandas (Colab o Jupyter)

Ruta de la clase y datos

- **Variables → calidad e imputación → entrenamiento / validación / prueba → Ridge y Lasso** antes y después de estandarizar → **K-Fold → estratificación → métricas**
- Datos: **300 viviendas simuladas** (precio en CLP) y **2.000 casos de fraude simulados** (5 % de positivos); ambos se generan en el notebook de la profesora, no hay archivos que descargar
- Caso de repaso: **scoring de crédito de consumo** (100.000 clientes, 2 % de impago)
- Pregunta guía: ¿cómo sé que mi modelo aprendió y no solo memorizó?

Primera vez con scikit-learn

La API completa cabe en tres verbos: `fit` (aprender de los datos), `transform` (aplicar lo aprendido) y `predict` (predecir). Regla de oro: **fit solo con entrenamiento**. Resumen en la página 7.

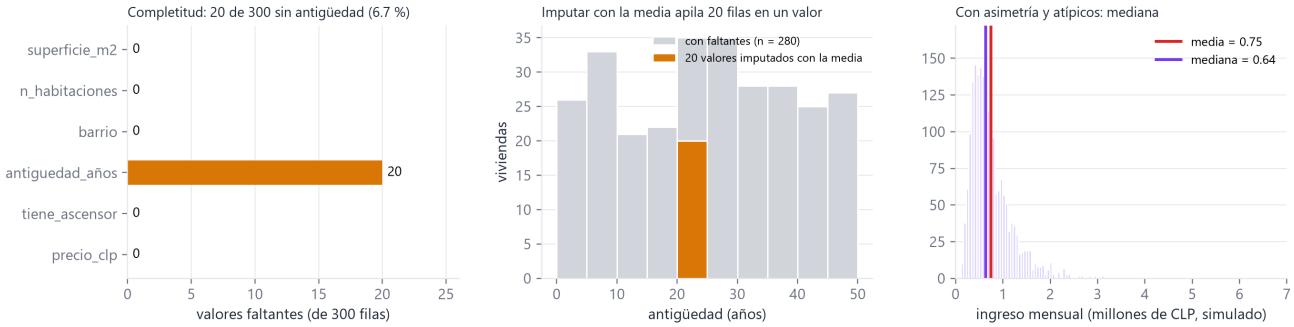
Clase 30 · Variables y calidad de los datos

Fuente: transcripción de la clase + diapositivas + apunte + notebook y caso de la profesora Laura Sánchez

Los modelos solo entienden números: hay que saber qué tipo es cada variable y si los datos son confiables antes de modelar. Un dato faltante se completa (imputa) con una regla que depende del tipo y de la distribución.

Conceptos clave

1. **Tipos de variables.** Numéricas (continuas o discretas) y categóricas (nominales sin orden, ordinales con orden). `n_habitaciones` es numérica discreta; `barrio`, nominal.
2. **Codificar categóricas.** Antes de entrenar se convierten a números, por ejemplo con one-hot (`pd.get_dummies` u `OneHotEncoder`): una columna 0/1 por categoría.
3. **Imputar.** Normal y sin atípicos: **media**. Con atípicos o asimetría: **mediana**. Categórica: **moda**. Si se relaciona con otras columnas: un modelo de regresión. Si falta un tercio, no se imputa: se revisa la recolección.
4. **Cuatro dimensiones de calidad. Exactitud** (refleja la realidad), **completitud** (no faltan datos), **coherencia** (las fuentes concuerdan), **frescura** (está vigente). Pueden fallar varias a la vez.
5. **Los datos mandan.** Sesgos en los datos se vuelven sesgos del modelo (ejemplo de la clase: generadores de imágenes). Un buen modelo no repara datos malos.



Izquierda: faltantes de las 300 viviendas (solo antigüedad: 20). Centro: tras imputar con la media, 20 filas quedan apiladas en un mismo valor (la desviación baja de 14.19 a 13.71). Derecha: ingresos simulados con cola larga.

Fórmulas y reglas

imputación: media · mediana · moda · regresión

% faltante = faltantes / filas = 20/300 = 6.7 %

media imputada □ la desviación estándar baja

imputer.fit(solo entrenamiento)

En Python

```
viviendas.dtypes · .isna().sum()
pd.get_dummies(df, columns=['barrio'])
SimpleImputer(strategy='median')
# 'mean' · 'median' · 'most_frequent'
```

Con los datos de la clase

- 300 viviendas, 6 columnas; antigüedad con 20 faltantes (6.7 %), media observada 24.56 años
- Ingresos simulados con cola: media 0.75 M vs mediana 0.64 M de CLP (los atípicos tiran la media)
- Caso del banco: ingreso (8 % nulos, sesgo a la derecha) → mediana; edad (2 %, simétrica) → media; comuna (5 %) → moda

Cuidado
La media y la mediana no se calculan con todo el dataset antes de dividir: se aprenden con el entrenamiento (fuga de datos, página 7).

Clase 30 · División de los datos y sobreajuste

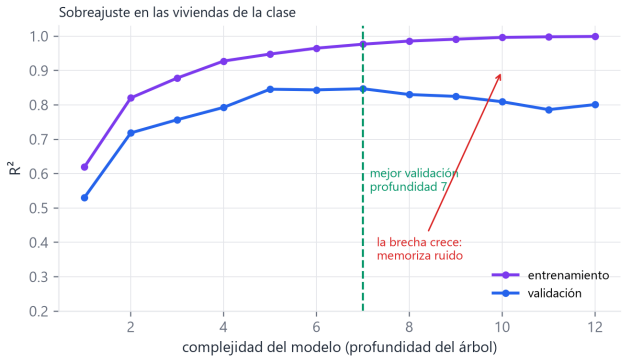
Fuente: transcripción de la clase + diapositivas + apunte + notebook y caso de la profesora Laura Sánchez

Se divide el dataset en tres: entrenamiento (aprende parámetros), validación (compara alternativas y ajusta hiperparámetros) y prueba (veredicto final, se usa una sola vez). Así se detecta el sobreajuste.

Conceptos clave

1. **Tres roles distintos.** Entrenamiento: el modelo aprende. Validación: tú eliges (alpha, profundidad, modelo). Prueba: se reserva y se mira al final; por eso es «sagrada».
2. **Sobreajuste (overfitting).** El modelo aprende ruido y particularidades del entrenamiento: rinde muy bien ahí y peor con datos nuevos. La señal es la **brecha** entre entrenamiento y validación.
3. **Subajuste (underfitting).** Modelo demasiado simple: rinde mal en ambos. El punto bueno está en el medio, donde la validación es máxima.
4. **Proporciones.** 70/15/15 es solo una referencia; depende del tamaño del problema. Con pocos datos se usa validación cruzada.
5. **Datos con tiempo.** Si hay orden temporal, se divide cronológicamente (entrenar con lo anterior, probar con lo posterior): un corte al azar filtra el futuro.





Izquierda: la división de la clase (210 / 45 / 45). Derecha: árboles de profundidad creciente sobre esas viviendas; el entrenamiento llega a 0.999 y la validación se estanca en ≈ 0.80 tras su máximo (0.847).

Fórmulas y reglas

n_entrenamiento = 0.70 · 300 = 210

n_temp = 0.30 · 300 = 90 → 45 y 45

brecha = métrica_entrenamiento − métrica_validación

la prueba no decide nada: solo informa

En Python

```
X_train, X_temp, y_train, y_temp =
    train_test_split(X, y, test_size=0.30,
                     random_state=42)
X_val, X_test, y_val, y_test =
    train_test_split(X_temp, y_temp,
test_size=0.5)
```

Con los datos de la clase

- Dos llamadas a train_test_split: 70 % y luego el 30 % restante a la mitad → 210 / 45 / 45
- Árbol de profundidad 12: R^2 0.999 en entrenamiento y 0.801 en validación; el mejor punto fue profundidad 7 (0.847)
- Caso del banco: el 90/10 del equipo de TI no deja dónde ajustar sin tocar la prueba
- En 120 de 300 particiones simples el R^2 de validación superó al de entrenamiento (≈ 45 filas de validación)

Cuidado
En clase se dijo que una validación mejor que el entrenamiento «es raro». Con pocas filas es normal; lo preocupante es una brecha grande a favor del entrenamiento.

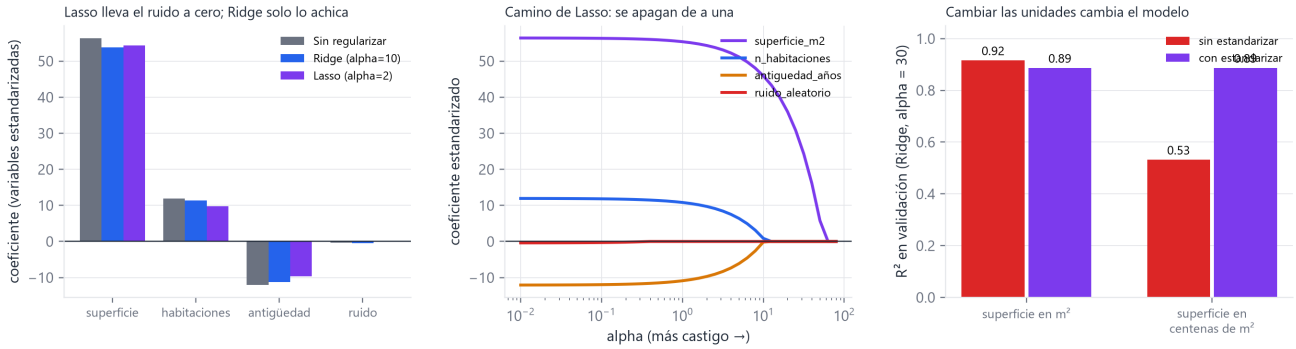
Clase 30 · Regularización: Ridge, Lasso y estandarizar

Fuente: transcripción de la clase + diapositivas + apunte + notebook y caso de la profesora Laura Sánchez

Regularizar es castigar la complejidad: se suma una penalización a los coeficientes. Como el castigo depende del tamaño de cada coeficiente, las variables deben estar en una escala comparable (estandarizar).

Conceptos clave

1. **Ridge (L2).** Penaliza la suma de coeficientes al cuadrado: **los achica**, normalmente sin llegar a cero. Conserva todas las variables.
2. **Lasso (L1).** Penaliza la suma de valores absolutos: puede llevar coeficientes **exactamente a cero** y actúa como selección de variables. Elastic Net mezcla ambos.
3. **alpha.** Es un hiperparámetro: mayor alpha, más castigo. Se elige con validación, no con la prueba. Que Lasso anule una variable depende de alpha y de los datos.
4. **Estandarizar.** Restar la media y dividir por la desviación estándar (z-score). Sin eso, el castigo depende de las **unidades**: una variable en cientos de m² paga mucho más que una en m².
5. **No arregla un modelo malo.** Solo hace comparable la penalización. Y el StandardScaler se aprende con el entrenamiento, dentro de un Pipeline.



Izquierda: coeficientes del modelo estandarizado (la prueba de la profesora). Centro: camino de Lasso, el ruido se anula primero (alpha ≈ 0.4). Derecha: la misma regresión Ridge con la superficie en m² o en centenas de m².

Fórmulas y reglas

Ridge: minimiza RSS + $\alpha \cdot \Sigma\, \beta_j^2$

Lasso: minimiza RSS + $\alpha \cdot \Sigma\, |\beta_j|$

z = (x − media) / desviación estándar

α = 0 □ regresión sin regularizar

En Python

```
make_pipeline(StandardScaler(),
              Ridge(alpha=10))
make_pipeline(StandardScaler(),
              Lasso(alpha=2))
modelo[-1].coef_ # coeficientes
```

Con los datos de la clase

- Estandarizado: superficie 56.4 (sin regularizar) → 53.8 (Ridge) → 54.3 (Lasso); el ruido: −0.40, −0.44 y exactamente 0
- Sin estandarizar los coeficientes (2.18, 8.19, −0.86) no son comparables con los estandarizados (56.4, 11.9, −12.1): cambian las unidades
- Ridge con alpha = 30: superficie en m² R^2 0.916; en centenas de m² cae a 0.532; estandarizado siempre 0.887

Cuidado
Con alpha = 2 Lasso ya anula el ruido incluso sin estandarizar (−0.000): la demo de clase no muestra el daño de no escalar; el panel derecho sí.

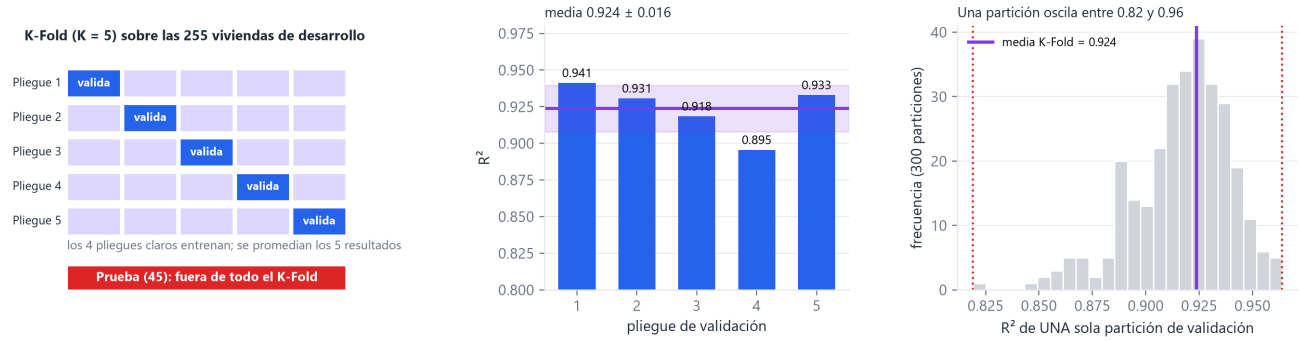
Clase 30 · Validación cruzada K-Fold y estratificación

Fuente: transcripción de la clase + diapositivas + apunte + notebook y caso de la profesora Laura Sánchez

Una sola partición puede ser «fácil» o «difícil» por azar. K-Fold repite la validación K veces y promedia; la estratificación conserva la proporción de clases al dividir. Ninguna reemplaza al conjunto de prueba.

Conceptos clave

1. **K-Fold.** Se parten los datos en K pliegues; cada uno actúa una vez como validación y los otros K − 1 entrenan. Se promedian los K resultados.
2. **Media y dispersión.** No solo importa el promedio: la desviación entre pliegues dice cuán estable es el modelo. Dispersión grande: el resultado depende de la muestra.
3. **K-Fold sirve en el desarrollo.** Se hace con los datos de entrenamiento (o entrenamiento + validación). La prueba queda fuera hasta el final.
4. **Muestreo estratificado.** Con clases raras (5 % de fraude) conserva la proporción en cada partición. No aumenta los fraudes: solo evita que una partición quede casi sin ellos.
5. **Otros muestreos.** Aleatorio simple (sin estructura ni desbalance) y consciente del tiempo (orden cronológico).



Izquierda: esquema de 5 pliegues (la prueba queda aparte). Centro: R^2 por pliegue con la media y ± una desviación. Derecha: R^2 de 300 particiones simples; una sola puede dar de 0.82 a 0.96.

Fórmulas y reglas

tamaño del pliegue = n / K

resultado = promedio de los K puntajes

dispersión = desviación entre pliegues

stratify=y □ misma proporción de clases

En Python

```
kf = KFold(n_splits=5, shuffle=True,
           random_state=42)
cross_val_score(pipe, X_dev, y_dev,
                cv=kf, scoring='r2')
train_test_split(X, y, stratify=y)
```

Con los datos de la clase

- 255 viviendas de desarrollo, 5 pliegues: R^2 0.941 · 0.931 · 0.918 · 0.895 · 0.933 → media 0.924, desviación 0.016
- Fraude: 109 positivos en 2.000 (5.45 %); entrenamiento 5.467 %, prueba 5.400 % con stratify
- Sin estratificar (n = 2.000) la prueba varía de 2.8 % a 8.4 %; con n = 200 hay 2.2 % de particiones sin ningún fraude en la prueba

Cuidado
El notebook de la clase hace el K-Fold con las 300 filas (0.921 ± 0.010), o sea, incluye la prueba. Aquí se usa solo entrenamiento + validación. En clasificación desbalanceada usa `StratifiedKFold`.

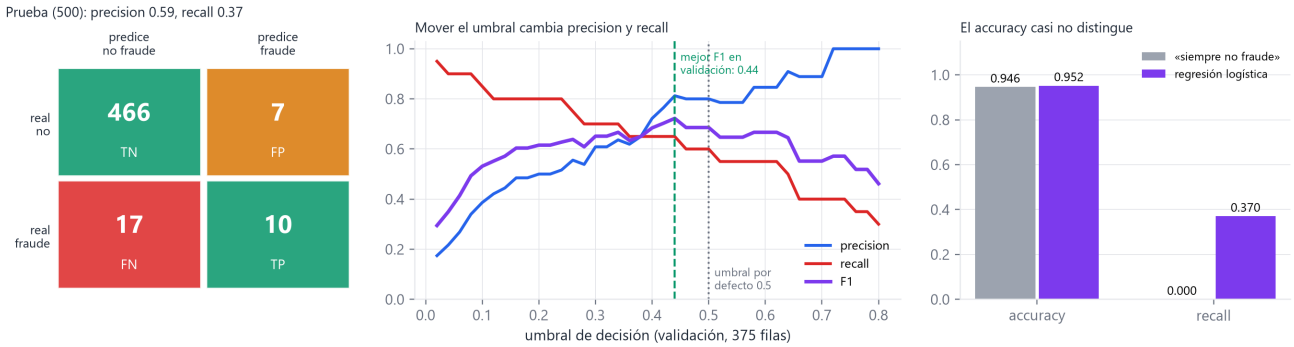
Clase 30 · Métricas de evaluación

Fuente: transcripción de la clase + diapositivas + apunte + notebook y caso de la profesora Laura Sánchez

Con clases desbalanceadas el accuracy engaña: un modelo que siempre dice «no fraude» acierta el 94.6 % y no detecta nada. Se mira la matriz de confusión y precision, recall y F1, según qué error cuesta más.

Conceptos clave

1. **Matriz de confusión.** TP (fraude detectado), FP (falsa alarma), FN (fraude que se escapó) y TN. Todas las métricas salen de estas cuatro cifras.
2. **Precision.** De todo lo que marqué como positivo, ¿cuánto lo era? Mide la confiabilidad de las alertas: precision alta = pocas falsas alarmas.
3. **Recall (exhaustividad).** De todos los positivos reales, ¿cuántos detecté? Recall alto = se escapan pocos. Mismo numerador que precision, **otro total**.
4. **F1.** Media armónica de precision y recall: resume el equilibrio cuando ambos errores importan. Subir una suele bajar la otra (umbral).
5. **Regresión.** MAE (error medio en las unidades del precio), RMSE (castiga más los errores grandes) y R^2 (proporción de variabilidad explicada). Un R^2 alto no basta.



Izquierda: matriz de confusión de la regresión logística en la prueba (500 casos). Centro: precision, recall y F1 al variar el umbral, calculados en validación. Derecha: accuracy de «siempre no fraude» frente al modelo.

Fórmulas y reglas

accuracy = (TP + TN) / n

precision = TP / (TP + FP)

recall = TP / (TP + FN)

F1 = 2·P·R / (P + R)

En Python

```
precision_score(y, pred)
recall_score(y, pred)
f1_score(y, pred)
confusion_matrix(y, pred)
modelo.predict_proba(X)[:, 1] >= umbral
```

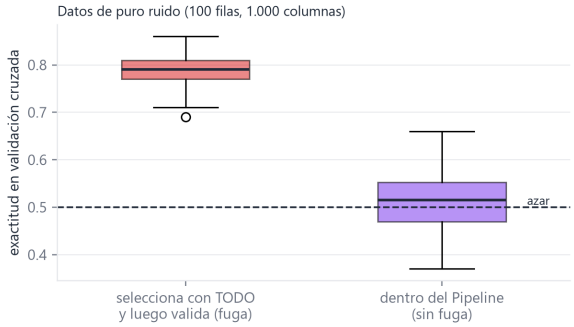
Con los datos de la clase

- Matriz: TN 466 · FP 7 · FN 17 · TP 10 → accuracy 0.952, precision 0.588, recall 0.370, F1 0.455
- «Siempre no fraude»: accuracy 0.946 y recall 0: el 0.952 apenas lo supera
- Regresión de precios (prueba): MAE 13.9 M, RMSE 16.2 M de CLP, R^2 0.904
- Caso del banco (2 % de impago): un falso negativo es costoso, así que importa el recall, sin olvidar la precision

Cuidado
En clase se dijo F1 «50 %»; la salida real del notebook es 0.455. Con 27 fraudes en la prueba, mover el umbral cambia mucho el resultado: elígelo en validación.

scikit-learn, Pipeline y fuga de datos

Lo que más se rompe en la primera clase de Machine Learning



MAL escalar / imputar / seleccionar con todas las filas → **luego dividir**

BIEN dividir primero → **fit solo con train → transform en val/prueba**

Pipeline = el «dividir primero» automático dentro de cada pliegue.

Datos de puro ruido: con 1.000 columnas sin relación alguna con la etiqueta, seleccionar las 10 «mejores» con todas las filas y luego validar da ≈ 0.79 de exactitud (40 repeticiones); hacerlo dentro de un Pipeline da ≈ 0.51, que es lo honesto.

Verbo	Qué hace	Con qué datos	Ejemplo
fit	Aprende (media, coeficientes, categorías…)	**solo entrenamiento**	scaler.fit(X_train)
transform	Aplica lo aprendido	entrenamiento, validación, prueba	scaler.transform(X_val)
fit_transform	fit + transform seguidos	solo entrenamiento	scaler.fit_transform(X_train)
predict	Predice con un modelo ya ajustado	validación, prueba, datos nuevos	modelo.predict(X_test)
Pipeline	Encadena pasos: hace fit de cada uno solo con lo que recibe	se llama con `.fit(X_train, y_train)`	make_pipeline(StandardScaler(), Ridge())
ColumnTransformer	Un tratamiento por tipo de columna	dentro del Pipeline	num → imputar + escalar; cat → moda + one-hot

Fuga de datos (data leakage)
Es usar, directa o indirectamente, información del conjunto de prueba (o de la validación) durante el desarrollo: para entrenar, imputar, escalar, seleccionar variables o elegir hiperparámetros. El resultado se ve mejor de lo que será en producción. La profesora lo menciona en el caso de repaso; aquí la forma práctica de evitarlo es el **Pipeline**.

	Checklist del flujo	Cómo sé que lo hice bien
☐	Divido primero	antes de imputar, escalar, codificar o seleccionar
☐	Uso un Pipeline	así cada pliegue de K-Fold re-aprende sus propios pasos
☐	Estratifico si hay clases raras	stratify=y y StratifiedKFold
☐	Elijo hiperparámetros con validación o K-Fold	alpha, profundidad, umbral
☐	Miro la prueba una sola vez	con el modelo ya decidido
☐	Reporto varias métricas	accuracy no basta con clases desbalanceadas

Alertas del material, repaso y criterio de dominio

Diferencias entre materiales y qué conviene corregir (verificado con código)

Notebook, diapositivas y apunte

- **K-Fold con las 300 filas** (Bloque 5): `cross_val_score(modelo, X, y)` incluye la prueba, aunque el texto dice que sigue reservada. Con entrenamiento + validación: 0.924 ± 0.016; con las 300: 0.921 ± 0.010.
- **La validación no se usa**: `X_val` se crea y no se vuelve a tocar; el clasificador de fraude se evalúa directo en la prueba, sin validar antes.
- **«Sin estandarizar es difícil llevar a cero»** (clase): con alpha = 2, Lasso anula el ruido en ambos casos. El daño real es de unidades: Ridge (alpha 30) pasa de R^2 0.916 a 0.532 si la superficie se mide en centenas de m^2. Además las dos tablas de coeficientes tienen unidades distintas: no se pueden comparar entre sí.
- **Diapositiva «Prueba (61 %)»**: la curva de la brecha 98 % → 61 % es ilustrativa y es de **validación** (la profesora lo corrige en clase: «esto no es prueba, es validación»).
- **COVID y la prueba**: «el conjunto de prueba simula ese futuro desconocido». Una prueba tomada al azar del mismo periodo **no** detecta un cambio de contexto; para eso hace falta una división temporal (el apunte sí lo matiza).
- **«Validación mejor que entrenamiento = algo falla»**: con 45 filas ocurre en el 40 % de las particiones (120 de 300); es una señal para revisar, no un error.
- **Texto vs código**: `n_habitaciones` se describe entre 1 y 6 y el código genera 1 a 5.
- **F1 «50 %»** dicho en clase; la salida real es 0.455. **K-Fold:** `.std()` de NumPy usa ddof = 0 (0.016; con ddof = 1, 0.018) y los pliegues no son independientes.

Caso de repaso (scoring de crédito)

- **Pregunta 1.2**: la respuesta oficial dice **Exactitud**. Con las definiciones del propio apunte (coherencia: «las fuentes concuerdan»), que la web y el contrato no coincidan es primero un problema de **coherencia**; la exactitud entra al elegir cuál es la verdad (el contrato). Conviene responder ambas y justificar.
- **Pregunta 1.1**: imputar con mediana, media o moda es correcto, pero el valor se calcula solo con el entrenamiento; con 5 % de comunas vacías también cabe una categoría «desconocida».
- **Pregunta 3.3**: recall es la respuesta esperada, pero un banco real también fija un umbral con validación y mira precision, el costo de rechazar a un buen pagador y métricas como PR-AUC.

Transcripción con ruido

Whisper confundió Ridge (Rich, reach, pitch), Lasso (laso, lazo), overfitting (overpicking), exactitud (exagitu), exhaustividad (exageridad), K-Fold (casco holding) y DALL-E. Corregido: «200/45/35 viviendas» era 210/45/45. Hay texto irrecuperable hacia 0:28–0:55, un recreo hacia 1:12–1:24 y bucles de repetición, ya omitidos en el .txt y el .srt corregidos.

Preguntas de repaso

#	Pregunta	Respuesta corta
1	¿Para qué sirve cada conjunto: entrenamiento, validación y prueba?	Aprender parámetros; elegir alpha o modelo; veredicto final una sola vez.
2	¿Por qué el accuracy engaña con 95 % de no fraude?	Un modelo que siempre dice «no» ya acierta 95 % y su recall es 0.
3	¿Qué diferencia hay entre Ridge y Lasso y por qué estandarizar?	Ridge achica, Lasso puede anular; el castigo depende de las unidades.

Criterio de dominio para pasar a la siguiente clase

- ☐ Clasifico variables y elijo media, mediana o moda con una justificación
- ☐ Calculo a mano precision, recall, F1 y accuracy desde una matriz de confusión
- ☐ Explico por qué se estandariza antes de Ridge o Lasso y por qué el scaler se ajusta solo con entrenamiento
- ☐ Calculo el promedio y la desviación de un K-Fold y reparto una clase rara con estratificación
- ☐ Escribo un Pipeline con ColumnTransformer sin filtrar información de la prueba
- ☐ Resuelvo los bloques 1 a 8 del cuaderno con al menos 80 % de aciertos